# Welcome to Code Crunch!

## C1|Crunch Convos

**WED 2-3PM Weekly | Spring 2025**

### Attendance

**Earn a Certificate of Completion by attending at least 70% of the workshops**

linktr.ee/CODE.CRUNCH

# Unit #1

## C1| Crunch Convos: Python Fundamentals

# Python

High-level programming language.

- **Simple Syntax:** reads like English, little to no punctuation, indented, intuitive
- **Interpreted Language:** executes code line-by-line.
- **Versatile and General-Purpose:** used for web development, data analysis, AI, and more.

# Variables

A variable is a container that holds data.

- **Dynamic Typing**: You don't need to specify the type of a variable (like int or string) when you create it. Python automatically figures it out.
- **Reassignment is Flexible**: You can change the type of data a variable holds at any time. This makes Python easy to work with but requires careful tracking of your variable types.
- **Type Hinting** attempts to mitigate this.

```
x = 5                  # integer
name = "John"          # string
price = 19.99          # float
is_active = True # boolean

y : int = 10 # Type hinting
```

# Data Types

Types of data that a variable can hold.

- **Integer (int)**: Whole numbers
- **Float (float)**: Numbers with a decimal point
- **String (str)**: Text enclosed in quotes
- **Boolean (bool)**: True or False values
- **List (list)**: Ordered collection of items
- **Tuple (tuple)**: Ordered, immutable collection of items
- **Set (set)**: Unordered collection of unique items
- **Dictionary (dict)**: Key-value pairs (e.g., {'key': 'value'}).

```
x: int = 10
pi: float = 3.14159
name: str = "Alice"
is_valid: bool = True
nums: list[int] = [1,2,3,4,5]
coords: tuple = (1, 2)
grades : dict = {“a” : 90, “b” : 80}
```

# Printing

**Printing Statements**: print() displays text or variables.

**F-strings**: allow embedding variables and expressions directly within a string using {} and prefixed with f.

**Customizing the Separator and End Character**
By default, print() separates multiple items with a space and ends with a newline (\n).
Customize these using the sep and end parameters.

**Formatting Numbers**
You can format numbers using f-strings for alignment, padding, or decimals.

```python
# Print statement
a = "Hello"
b = "World"
print(a, b)

# f-strings
name = "Alice"
age = 21
print(f"My name is {name}, and I am {age} years old.")

# Separators
print("apple", "banana", "cherry", sep="-")
print("Hello", end=" ")
print("World!")

# Formatting
value = 1234.56789
print(f"Value: {value:.2f}")
print(f"{value:10.2f}")
```

# Conditional Logic

**Conditional Statements:** Used to perform different actions based on different conditions.

**Syntax:** if, elif (else if) , and else keywords.

- if condition1: Executes a block of code if a condition is true.
- elif condition2 : Checks another condition if the previous one is false.
- else : Executes a block of code if all other conditions are false.

```
1   num = 10
2
3   if num > 10:
4       print("Greater than 10")
5   elif num == 10:
6       print("Equal to 10")
7   else:
8       print("Less than 10")
9
```

# Comparison Operators

| | |
|---|---|
| == | Equal |
| != | Not equal |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal |
| <= | Less than or equal |

```
age = 18
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teenager")
else:
    print("Child")
```

# Logical Operators

Logical Operators combine multiple conditional expressions.

| | |
|---|---|
| and | True if both conditions evaluate to True |
| or | True if only 1 of the two conditions evaluate to True. |
| not | Opposite value (True or False) of a condition |

```
age = 25
has_id = True
is_member = False

if age >= 18 and has_id:
    print("You are allowed to enter.")

if age >= 18 or is_member:
    print("You can enter the members'
section.")

if not is_member:
    print("You need to become a member.")
```

# Loops

Loops are used to execute a block of code repeatedly until a condition is met.

*for* loop: Iterates over a sequence (e.g., list, range).

*while* loop: Repeats a block of code as long as a condition is true.

```
1   # For loop: Iterating over a list
2   fruits = ["apple", "banana", "cherry"]
3   for fruit in fruits:
4       print(fruit)
5
6   # While loop: Counting down from 5
7   count = 5
8   while count > 0:
9       print(count)
10      count -= 1
11
```

# While loop

```
count = 0
while count < 5:
    print(count)
    count -= 1
```

| Assignment Operators | |
|---|---|
| += | **Adds** a value to the current value of the variable and **assigns the sum to the variable** |
| -= | **Subtracts** a value from the current value of the variable and **assigns the difference to the variable** |
| *= | **Multiplies** a value from the current value of the variable and **assigns the product to the variable** |
| /= | **Divides** a value from the current value of the variable and **assigns the quotient to the variable** |

# For Loop

```python
# Range Function
for i in range(5):
    print(f"Number: {i}")

# Enumerate
colors = ["red", "green", "blue"]
for i, c in enumerate(colors):
    print(f"idx: {i}, color: {c}")

# Reversed (loops from 5 to 1)
for i in reversed(range(1, 6)):
    print(f"Countdown: {i}")
```

**Range Function:**

- Iterate a specific number of times
- Default start index is 0, and step increment is 1

**Enumerate:**

- Lets you loop over a sequence while getting both the index and the item.

**Reversed:**

- Reverses a sequence, such as one generated by `range`

# Functions

A function is a block of reusable code that performs a specific task.

**Defining Functions:** Use the def keyword, followed by the function name and parentheses. Inside the parentheses, you can define parameters.

**Function Parameters:** Functions can take arguments to modify their behavior.

**Return Statement:** Functions can return values using the return keyword. If no return is specified, the function returns None

```
def greet() -> None:
  print("Hello, World!")

greet()  # Output: Hello, World!

def add_numbers(a, b) -> int:
  return a + b

result : int = add_numbers(5, 7)
print(result)  # Output: 12
```

# Error Handling

Mechanisms to handle exceptions and errors in code.

**try-except Block**: Wraps code that might raise an exception.

**else**: Code block that runs if no exceptions are raised.

**finally**: Code block that always runs, regardless of whether an exception was raised.

```
try:
    # Code that might raise an error
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print("Division successful!")
finally:
    print("Execution complete.")
```

# Lists

A list is an ordered collection of elements, which can hold different data types (integers, strings, etc.)

**Accessing List Elements:**

- Use indexing to access elements (starting from 0).
- Use negative indices to access elements from the end (starting from -1).

**Getting the Length of a List:**

- Use the len() function to find out how many elements are in a list.

**Checking if an Element is in a List:**

- Use the in operator to check if an item exists in a list.

```
fruits: list[str] = ["apple”, "banana", "cherry"]

numbers : list[int] = [1, 2, 3, 4, 5]

mixed : list[Optional] = [1, "hello", True]

print(fruits[0])        # Output: apple

print(fruits[-1])       # Output: cherry

n : int = len(fruits)   # n = 3

if "apple" in fruits:
    print("Apple is in the list!")
```

# Traversing a List

**Using a `for` loop with `range`:**

```
def get_average(lst):
    total = 0
    for i in range(len(lst)):
        total += lst[i]
    return total / len(lst)
```

- We use the `range()` function to get the index of each element.
- Access each list element using `lst[i]`

**Using an enhanced `for` loop:**

```
def get_average (lst):
    total = 0
    for num in lst:
        total += num
    return total / len(lst)
```

- The **enhanced** `for` loop iterates over each element directly.
- It doesn't need index access, making it more concise.

**Using a `while` loop:**

```
def get_average(lst):
    total = 0
    i = 0
    while i < len(lst):
        total += lst[i]
        i += 1
    return total / len(lst)
```

- The `while` loop continues as long as `i` is less than the length of the list.
- We manually increment the index `i`

# List Manipulation

**Adding Elements:**

- `append()` adds an element to the end of the list.
- `insert()` adds an element at a specified index.

**Removing Elements:**

- `remove()` deletes a specific element.
- `pop()` removes the last element and returns the updated list.
- `clear()` empties the entire list.

**Sorting and Reversing:**

- `sort()` arranges elements in ascending order.
- `reverse()` reverses the order of elements.

```
fruits = ["apple", "banana", "cherry"]

fruits.append("orange")
# Output: ["apple", "banana", "cherry”, "orange"]

fruits.insert(1, "mango")
# Output: ["apple", "mango", "banana", "cherry"]

fruits.remove("banana")
# Output: ["apple", "cherry"]

last_fruit = fruits.pop()
# Output: ["apple", "banana"]

fruits.clear()
# Output: []
```

# List Slicing

Slicing is a way to extract a portion (or "slice") of a sequence (such as a list or string) using the slice notation:

[start:stop:step]

- [start:stop] Extracts elements from the start index up to (but not including) the stop index.
- [start:stop:step] The step argument specifies the interval between elements.

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

slice_1 = fruits[1:3]
# Output: ['banana', 'cherry']

slice_2 = fruits[:3]
# Output: ['apple', 'banana', 'cherry’]

slice_3 = fruits[::2]
# Output: ['apple', 'cherry', 'elderberry’]

slice_4 = fruits[::-1]
# Output: ['elderberry', 'date', 'cherry', 'banana', 'apple']
```

# Comprehension

A concise way to create lists, dictionaries, or sets using expressions.

**List Comprehension**: [expression for item in iterable].

**Dictionary Comprehension**: {key: value for item in iterable}.

**Set Comprehension**: {expression for item in iterable}.

```
# List Comprehension
squared_numbers = [x**2 for x in range(5)]  # Output: [0, 1, 4, 9, 16]

# Dictionary Comprehension
squares = {x: x**2 for x in range(5)}  # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Set Comprehension
unique_chars = {char for char in "hello"}  # Output: {'e', 'h', 'l', 'o'}
```

# Classes and Objects

Classes and objects are fundamental building blocks of Object-Oriented programming.

- **Class**: A template defining attributes and methods used as a blueprint for creating objects..
- **Object**: An instance of a class.
- **Attributes**: Characteristics or properties of an object.
- **Methods**: Functions defined inside a class.

```python
class Dog:
    # Class attributes
    species = "Canis lupus familiaris"

    def __init__(self, name: str, age: int):
        # Instance attributes
        self.name = name
        self.age = age

    def bark(self):
        return "Woof!"

# Creating an object
my_dog = Dog(name="Rex", age=5)

# Accessing object attributes
print(my_dog.name)  # Output: Rex
print(my_dog.age)   # Output: 5

# Calling object methods
print(my_dog.bark())  # Output: Woof!
```

# Attendance

Join us for the upcoming weekly sessions this Spring 2025!

**Monday:** 2PM
**Tuesdays:** 1PM & 2PM
**Wednesdays:** 2PM
**Thursdays:** 1PM , 2PM & 3PM
**Friday:** 2PM & 3PM

RSVP LU.MA/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
**Go to our website:** go.fiu.edu/codecrunch